

 **BIG NEWS: Neptune is joining OpenAI!** → Read the message from our CEO 

Table of contents

Neptune Blog

Cross-Validation in Machine Learning: How to Do It Right



Vladimir Lyashenko,
Abhishek Jha

 10 min

 25th April, 2025

ML Model Development

In machine learning (ML), generalization usually refers to the ability of an algorithm to be effective across various inputs. It means that the ML model does not encounter performance degradation on the new inputs from the same distribution of the training data.

For human beings generalization is the most natural thing possible. We can classify on the fly. For example, we would definitely recognize a dog even if we didn't see this breed before. Nevertheless, it might be quite a challenge for an ML model. That's why checking the algorithm's ability to generalize is an important task that requires a lot of attention when building the model.

To do that, we use **Cross-Validation (CV)**.

In this article we will cover:

- What is Cross-Validation: definition, purpose of use and techniques

Table of contents

- How to use these techniques: sklearn
- Cross-Validation in Machine Learning: sklearn, CatBoost
- Cross-Validation in Deep Learning: Keras, PyTorch, MxNet
- Best practices and tips: time series, medical and financial data, images

What is cross-validation?

Cross-validation is a technique for evaluating a machine learning model and testing its performance. CV is commonly used in applied ML tasks. It helps to compare and select an appropriate model for the specific predictive modeling problem.

CV is easy to understand, easy to implement, and it tends to have a lower bias than other methods used to count the model's efficiency scores. All this makes cross-validation a powerful tool for selecting the best model for the specific task.

There are a lot of different techniques that may be used to **cross-validate** a model. Still, all of them have a similar algorithm:

1. Divide the dataset into two parts: one for training, other for testing
2. Train the model on the training set
3. Validate the model on the test set
4. Repeat 1-3 steps a couple of times. This number depends on the CV method that you are using



Home > Blog > ML Model Development

Search in Blog...

- Hold-out
- K-folds
- Leave-one-out
- Leave-p-out
- Stratified K-folds

- Repeated K-folds

Table of contents

Hold-out cross-validation

Hold-out cross-validation is the simplest and most common technique. You might not know that it is a **hold-out** method but you certainly use it every day.

The algorithm of hold-out technique:

1. Divide the dataset into two parts: the training set and the test set. Usually, 80% of the dataset goes to the training set and 20% to the test set but you may choose any splitting that suits you better
2. Train the model on the training set
3. Validate on the test set
4. Save the result of the validation



That's it.

We usually use the hold-out method on large datasets as it requires training the model only once.

It is really easy to implement hold-out. For example, you may do it using `sklearn.model_selection.train_test_split`.

```
import numpy as np
from sklearn.model_selection import train_test_split

X, y = np.arange(10).reshape((5, 2)), range(5)
X_train, X_test, y_train, y_test = train_test_split(X, y,

test_size=0.2,

random_state=111)
```

Still, hold-out has a major disadvantage.

Table of contents

represent the test set. Both training and test sets may differ a lot, one of them might be easier or harder.

Moreover, the fact that we test our model only once might be a bottleneck for this method. Due to the reasons mentioned before, the result obtained by the hold-out technique may be considered inaccurate.

k-Fold cross-validation

k-Fold cross-validation is a technique that minimizes the disadvantages of the hold-out method. k-Fold introduces a new way of splitting the dataset which helps to overcome the “test only once bottleneck”.

The algorithm of the k-Fold technique:



k-Fold cross validation

1. Pick a number of folds – k. Usually, k is 5 or 10 but you can choose any number which is less than the dataset’s length.
2. Split the dataset into k equal (if possible) parts (they are called folds)
3. Choose k – 1 folds as the training set. The remaining fold will be the test set
4. Train the model on the training set. On each iteration of cross-validation, you must train a new model independently of the model trained on the previous iteration
5. Validate on the test set
6. Save the result of the validation
7. Repeat steps 3 – 6 k times. Each time use the remaining fold as the test set. In the end, you should have validated the model on every fold that you have.
8. To get the final score average the results that you got on step 6.

To perform k-Fold cross-validation you can use `sklearn.model_selection.KFold`.

```
import numpy as np
```

Table of contents

```
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])  
y = np.array([1, 2, 3, 4])  
kf = KFold(n_splits=2)  
  
for train_index, test_index in kf.split(X):  
    print("TRAIN:", train_index, "TEST:", test_index)  
    X_train, X_test = X[train_index], X[test_index]  
    y_train, y_test = y[train_index], y[test_index]
```

In general, it is always better to use k-Fold technique instead of hold-out. In a head to head, comparison k-Fold gives a more stable and trustworthy result since training and testing is performed on several different parts of the dataset. We can make the overall score even more robust if we increase the number of folds to test the model on many different sub-datasets.

Still, k-Fold method has a disadvantage. Increasing k results in training more models and the training process might be really expensive and time-consuming.

Leave-one-out cross-validation

Leave-one-out cross-validation (LOOCV) is an extreme case of **k-Fold CV**.

Imagine if **k** is equal to **n** where **n** is the number of samples in the dataset. Such **k-Fold** case is equivalent to **Leave-one-out** technique.

The algorithm of LOOCV technique:

1. Choose one sample from the dataset which will be the test set
2. The remaining $n - 1$ samples will be the training set
3. Train the model on the training set. On each iteration, a new model must be trained
4. Validate on the test set
5. Save the result of the validation

- Repeat steps 1 – 5 n times as for n samples we have n different training and test sets.

Table of contents

For LOOCV sklearn also has a built-in method. It can be found in the `model_selection` library – `sklearn.model_selection.LeaveOneOut`.

```
import numpy as np
from sklearn.model_selection import LeaveOneOut

X = np.array([[1, 2], [3, 4]])
y = np.array([1, 2])
loo = LeaveOneOut()

for train_index, test_index in loo.split(X):
    print("TRAIN:", train_index, "TEST:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

The greatest advantage of Leave-one-out cross-validation is that it doesn't waste much data. We use only one sample from the whole dataset as a test set, whereas the rest is the training set. But when compared with k-Fold CV, LOOCV requires building n models instead of k models, when we know that n which stands for the number of samples in the dataset is much higher than k . It means LOOCV is more computationally expensive than k-Fold, it may take plenty of time to cross-validate the model using LOOCV.

Thus, the Data Science community has a general rule based on empirical evidence and different researches, which suggests that 5- or 10-fold cross-validation is the best choice.

Table of contents

Leave-p-out cross-validation

Leave-p-out cross-validation (LpOC) is similar to Leave-one-out CV as it creates all the possible training and test sets by using p samples as the test set. All mentioned about LOOCV is true and for LpOC.

Still, it is worth mentioning that unlike LOOCV and k-Fold test sets will overlap for LpOC if p is higher than 1.

The algorithm of LpOC technique:

1. Choose p samples from the dataset which will be the test set
2. The remaining $n - p$ samples will be the training set
3. Train the model on the training set. On each iteration, a new model must be trained
4. Validate on the test set
5. Save the result of the validation
6. Repeat steps 2 – 5 C_p^n times
7. To get the final score average the results that you got on step 5

You can perform Leave-p-out CV using sklearn – `sklearn.model_selection.LeavePOut`.

```
import numpy as np
from sklearn.model_selection import LeavePOut

X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
y = np.array([1, 2, 3, 4])
lpo = LeavePOut(2)

for train_index, test_index in lpo.split(X):
    print("TRAIN:", train_index, "TEST:", test_index)
```

```
X_train, X_test = X[train_index], X[test_index]  
y_train, y_test = y[train_index], y[test_index]
```

Table of contents

LpOC has all the disadvantages of the LOOCV, but, nevertheless, it's as robust as LOOCV.

Stratified k-Fold cross-validation

Sometimes we may face a large imbalance of the target value in the dataset. For example, in a dataset concerning wristwatch prices, there might be a larger number of wristwatch having a high price. In the case of classification, in cats and dogs dataset there might be a large shift towards the dog class.

Stratified k-Fold is a variation of the standard **k-Fold CV** technique which is designed to be effective in such cases of target imbalance.

It works as follows. Stratified k-Fold splits the dataset on k folds such that each fold contains approximately the same percentage of samples of each target class as the complete set. In the case of regression, Stratified k-Fold makes sure that the mean target value is approximately equal in all the folds.

The algorithm of Stratified k-Fold technique:

1. Pick a number of folds – k
2. Split the dataset into k folds. Each fold must contain approximately the same percentage of samples of each target class as the complete set
3. Choose k – 1 folds which will be the training set. The remaining fold will be the test set
4. Train the model on the training set. On each iteration a new model must be trained
5. Validate on the test set
6. Save the result of the validation
7. Repeat steps 3 – 6 k times. Each time use the remaining fold as the test set. In the end, you should have validated the model on every fold that you have.
8. To get the final score average the results that you got on step 6.

Table of contents

As you may have noticed, the algorithm for Stratified k-Fold technique is similar to the standard k-Folds. You don't need to code something additionally as the method will do everything necessary for you.

Stratified k-Fold also has a built-in method in sklearn – `sklearn.model_selection.StratifiedKFold`.

```
import numpy as np
from sklearn.model_selection import StratifiedKFold

X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
y = np.array([0, 0, 1, 1])
skf = StratifiedKFold(n_splits=2)

for train_index, test_index in skf.split(X, y):
    print("TRAIN:", train_index, "TEST:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

All mentioned above about k-Fold CV is true for Stratified k-Fold technique. When choosing between different CV methods, make sure you are using the proper one. For example, you might think that your model performs badly simply because you are using k-Fold CV to validate the model which was trained on the dataset with a [class imbalance](#). To avoid that you should always do a proper exploratory data analysis on your data.

Repeated k-Fold cross-validation

Table of contents

Repeated k-Fold cross-validation or **Repeated random sub-sampling CV** is probably the most robust of all **CV** techniques in this paper. It is a variation of **k-Fold** but in the case of **Repeated k-Folds** **k** is not the number of folds. It is the number of times we will train the model.

The general idea is that on every iteration we will randomly select samples all over the dataset as our test set. For example, if we decide that 20% of the dataset will be our test set, 20% of samples will be randomly selected and the rest 80% will become the training set.

The algorithm of Repeated k-Fold technique:

1. Pick k – number of times the model will be trained
2. Pick a number of samples which will be the test set
3. Split the dataset
4. Train on the training set. On each iteration of cross-validation, a new model must be trained
5. Validate on the test set
6. Save the result of the validation
7. Repeat steps 3-6 k times
8. To get the final score average the results that you got on step 6.

Repeated k-Fold has clear advantages over standard k-Fold CV. Firstly, the composition of train / test splits is not dependent on the number of iterations.

Table of contents

to selection bias.

Still, there are some disadvantages. k-Fold CV guarantees that the model will be tested on all samples, whereas Repeated k-Fold is based on randomization which means that some samples may never be selected to be in the test set at all. At the same time, some samples might be selected multiple times. Thus making it a bad choice for imbalanced datasets.

Sklearn will help you to implement a Repeated k-Fold CV. Just use `sklearn.model_selection.RepeatedKFold`. In sklearn implementation of this technique you must set the number of folds that you want to have (`n_splits`) and the number of times the split will be performed (`n_repeats`). It guarantees that you will have different folds on each iteration.

```
import numpy as np
from sklearn.model_selection import RepeatedKFold

X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
y = np.array([0, 0, 1, 1])
rkf = RepeatedKFold(n_splits=2, n_repeats=2, random_state=42)

for train_index, test_index in rkf.split(X):
    print("TRAIN:", train_index, "TEST:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

Nested k-Fold

Unlike the other CV techniques, which are designed to evaluate the quality of an algorithm, **Nested k-fold CV** is used to train a model in which hyperparameters also need to be optimized. It estimates the generalization error of the underlying model and its (hyper)parameter search.

Nested k-Fold cross-

Table of contents

The algorithm of Nested k-Fold technique:

1. Define set of hyper-parameter combinations, C , for current model. If model has no hyper-parameters, C is the empty set.
2. Divide data into K folds with approximately equal distribution of cases and controls.
3. (outer loop) For fold k , in the K folds:
 - Set fold k , as the test set.
 - Perform automated feature selection on the remaining $K-1$ folds.
 - For parameter combination c in C :
 - (inner loop) For fold k , in the remaining $K-1$ folds:
 - Set fold k , as the validation set.
 - Train model on remaining $K-2$ folds.
 - Evaluate model performance on fold k .
 - Calculate average performance over $K-2$ folds for parameter combination c .
 - Train model on $K-1$ folds using hyper-parameter combination that yielded best average performance over all steps of the inner loop.
 - Evaluate model performance on fold k .
4. Calculate average performance over K folds.

The inner loop performs cross-validation to identify the best features and model hyper-parameters using the $k-1$ data folds available at each iteration of the outer loop. The model is trained once for each outer loop step and evaluated on the held-out data fold. This process yields k evaluations of the model performance, one for each data fold, and allows the model to be tested on every sample.

It is to be noted that this technique is computationally expensive because plenty of models is trained and evaluated. Unfortunately, there is no built-in method in sklearn that would perform Nested k-Fold CV for you.

You can either implement it yourself or refer to the implementation [here](#).

Time-series cross-validation

Table of contents

Traditional cross-validation techniques don't work on sequential data such as time-series because we cannot choose random data points and assign them to either the test set or the train set as it makes no sense to use the values from the future to forecast values in the past. There are mainly two ways to go about this:

1. Rolling cross-validation

Cross-validation is done on a rolling basis i.e. starting with a small subset of data for training purposes, predicting the future values, and then checking the accuracy on the forecasted data points. The following image can help you get the intuition behind this approach.

Rolling
cross-
validation
| [Source](#)

2. Blocked cross-validation

The first technique may introduce leakage from future data to the model. The model will observe future patterns to forecast and try to memorize them. That's why blocked cross-validation was introduced.

Blocked
cross-
validation
| [Source](#)

It works by adding margins at two positions. The first is between the training and validation folds in order to prevent the model from observing lag values which are used twice, once as a regressor and another as a response. The second is between the folds used at each iteration in order to prevent the model from memorizing patterns from one iteration to the next.

Cross-validation in Machine Learning

When is cross-validation the right choice?

Table of contents

absolute necessity:

1. Limited dataset

Let's say we have 100 data points and we are dealing with a multi-class classification problem with 10 classes, this averages out to ~10 examples per class. In an 80-20 train-test split, this number would go down even further to 8 samples per class for training. The smart thing to do here would be to use cross-validation and utilize our entire dataset for training as well as testing.

Read also

[Leveraging Unlabeled Image Data With Self-Supervised Learning or Pseudo Labeling With Mateusz Opala](#)

2. Dependent data points

When we perform a random train-test split of our data, we assume that our examples are independent. It means that knowing some instances will not help us understand other instances. However, that's not always the case, and in such situations, it's important that our model gets familiar with the entire dataset which is possible with cross-validation.

3. Cons of single metric

In the absence of cross-validation, we only get a single value of accuracy or precision or recall which could be an outcome of chance. When we train multiple models, we eliminate such possibilities and get a metric per model which results in robust insights.

4. Hyperparameter tuning

Although there are many [methods to tune the hyperparameters](#) of your model such as grid search, Bayesian optimization, etc., this exercise can't be done on training or test set, and a need for a validation set arises. Thus, we fall back to

the same splitting problem that we have discussed above and cross-validation can help us out of this.

Table of contents

May be useful

How to Manage, Track, and Visualize Hyperparameters of Machine Learning Models?

Cross-validation in Deep Learning

Cross-validation in Deep Learning (DL) might be a little tricky because most of the CV techniques require training the model at least a couple of times.

In deep learning, you would normally tempt to avoid CV because of the cost associated with training k different models. Instead of doing k-Fold or other CV techniques, you might use a random subset of your training data as a hold-out for validation purposes.

For example, **Keras** deep learning library allows you to pass one of two parameters for the fit function that performs training.

1. **validation_split**: percentage of the data that should be held out for validation
2. **validation_data**: a tuple of (X, y) which should be used for validation. This parameter overrides the validation_split parameter which means you can use only one of these parameters at once.

The same approach is used in official tutorials of other DL frameworks such as PyTorch and MxNet. They also suggest splitting the dataset into three parts: training, validation, and testing.

1. Training – a part of the dataset to train on
2. Validation – a part of the dataset to validate on while training
3. Testing – a part of the dataset for final validation of the model

Still, you can use cross-validation in DL tasks if the dataset is tiny (contains hundreds of samples). In this case, learning a complex model might be an irrelevant task so make sure that you don't complicate the task further.

Best practices and tips

Table of contents

It's worth mentioning that sometimes performing cross-validation might be a little tricky.

For example, it's quite easy to make a logical mistake when splitting the dataset which may lead to an untrustworthy CV result.

You may find some tips that you need to keep in mind when cross-validating a model below:

1. Be logical when splitting the data (does the splitting method make sense)
2. Use the proper CV method (is this method viable for my use-case)
3. When working with time series don't validate on the past (see the first tip)
4. When working with medical or financial data remember to split by person. Avoid having data for one person both in the training and the test set as it may be considered as data leak
5. When cropping patches from larger images remember to split by the large image Id

Of course, tips differ from task to task and it's almost impossible to cover all of them. That's why performing a **solid exploratory data analysis** before starting to cross-validate a model is always the best practice.

Final thoughts

Cross-validation is a powerful tool. Every Data Scientist should be familiar with it. In real life, you can't finish the project without cross-validating a model.

In my opinion, the best **CV** techniques are **Nested k-Fold** and standard **k-Fold**. Personally, I used them in the task of Fraud Detection. **Nested k-Fold**, as well as **GridSearchCV**, helped me to tune the parameters of my model. **k-Fold** on the other hand was used to evaluate my model's performance.

In this article, we have figured out what cross-validation is, what CV techniques are there in the wild, and how to implement them. In the future ML algorithms will definitely perform even better than today. Still, cross-validation will always be needed to back your results up.

Hopefully, with this information, you will have no problems setting up the CV for your machine learning project!

Table of contents

Resources

1. <https://www.geeksforgeeks.org/cross-validation-machine-learning/>
2. <https://machinelearningmastery.com/k-fold-cross-validation/>
3. <https://towardsdatascience.com/cross-validation-in-machine-learning-72924a69872f>
4. <https://towardsdatascience.com/why-and-how-to-do-cross-validation-for-machine-learning-d5bd7e60c189>
5. https://scikit-learn.org/stable/modules/cross_validation.html

Was the article useful?



Yes



No

Suggest changes

More about Cross-Validation in Machine Learning: How to Do It Right

Check out our **product resources** and **related articles** below:

 Related article

We are joining OpenAI

[Read more](#) →

 Related article

Synthetic Data for LLM Training

[Read more](#) →

 Related article

 Related article

What are LLM Embeddings: All
you Need to Know

Detecting and Fixing 'Dead
Neurons' in Foundation Models

Table of contents

Explore more content topics:

[Computer Vision](#)[General](#)[LLMOps](#)[ML Model Development](#)[ML Tools](#)[MLOps](#)[Natural Language Processing](#)[Paper Reflections](#)[Reinforcement Learning](#)[Tabular Data](#)[Time Series](#)

Newsletter

Top articles, case studies, events (and more) in your
inbox every month.

[Get Newsletter](#)

PRODUCT

COMPARE

COMMUNITY

COMPANY

Table of contents

[Terms of Service](#) [Privacy Policy](#) [SLA](#)

Copyright © 2025 Neptune Labs. All rights reserved.